

/home/sukesh273/hashtocoin-1.0.0

sukesh273@LAPTOP-PI08CKLL:~\$ pwd

/home/sukesh273

sukesh273@LAPTOP-PI08CKLL:~\$ mkdir -p ~/OS-Second-Program

sukesh273@LAPTOP-PI08CKLL:~\$ cd ~/OS-Second-Program

sukesh273@LAPTOP-PI08CKLL:~/OS-Second-Program\$ pwd

/home/sukesh273/OS-Second-Program

sukesh273@LAPTOP-PI08CKLL:~/OS-Second-Program\$ code .

sukesh273@LAPTOP-PI08CKLL:~/OS-Second-Program\$

ExplorerWelcome X program2.py X

OS-S...program2.py

program2.py

```
1 processes = [
2     {"pid": "P1", "arrival": 0, "burst": 7},
3     {"pid": "P2", "arrival": 2, "burst": 4},
4     {"pid": "P3", "arrival": 4, "burst": 1},
5     {"pid": "P4", "arrival": 5, "burst": 4},
6 ]
7
8 # PART A: FCFS (First Come First Serve)
9
10 def fcfs(process_list):
11     current_time = 0
12     intervals = []
13
14     ordered = sorted(
15         process_list,
16         key=lambda p: (p["arrival"], p["pid"])
17     )
18
19     for process in ordered:
20         if current_time < process["arrival"]:
21             intervals.append(("IDLE", current_time, process["arrival"]))
22             current_time = process["arrival"]
23
24         start = current_time
25         end = start + process["burst"]
26         intervals.append((process["pid"], start, end))
27         current_time = end
28
29     return intervals
```

> Outline

Ln 1

Ubuntu-24.04 0 0 0

```
# PART B: NON-PREEMPTIVE SJF
```

```
def sjf(process_list):  
    remaining = process_list.copy()  
  
    current_time = 0  
    intervals = []  
  
    while remaining:  
        ready = [  
            p for p in remaining  
            if p["arrival"] <= current_time  
        ]  
  
        if not ready:  
            next_arrival = min(p["arrival"] for p in remaining)  
  
            intervals.append(  
                ("IDLE", current_time, next_arrival)  
            )  
  
            current_time = next_arrival  
            continue  
  
        process = min(  
            ready,  
            key=lambda p: (p["burst"], p["arrival"], p["pid"])  
        )
```

```

        start = current_time
        end = start + process["burst"]

        intervals.append(
            (process["pid"], start, end)
        )

        current_time = end
        remaining.remove(process)

    return intervals

# =====
# PART C: DISPLAY RESULTS
# =====

def show_result(title, intervals):
    print("\n" + title)
    print("Process Start End")

    sequence = []

    for pid, start, end in intervals:
        print(f"{pid:<9}{start:<7}{end}")

```

```

✓   for pid, start, end in intervals:
      print(f"{pid:<9}{start:<7}{end}")

✓   if pid != "IDLE":
      sequence.append(pid)

      print("Sequence:", "->".join(sequence))

# FINAL PYTHON LINES

print("INPUT PROCESSES")
print("PID AT BT")

✓ for p in processes:
    print(f'{p["pid"]:<5} {p["arrival"]:<4} {p["burst"]}')

show_result("FCFS SCHEDULING", fcfs(processes))
show_result("SJF SCHEDULING", sjf(processes))

```

```
● suresh273@LAPTOP-PI08CKLL:~/OS-Second-Program$ pwd
/home/suresh273/OS-Second-Program
● suresh273@LAPTOP-PI08CKLL:~/OS-Second-Program$ python3 program2.py
INPUT PROCESSES
PID AT BT
P1    0    7
P2    2    4
P3    4    1
P4    5    4

FCFS SCHEDULING
Process Start End
P1      0     7
P2      7    11
P3     11    12
P4     12    16
Sequence: P1->P2->P3->P4

SJF SCHEDULING
Process Start End
P1      0     7
P3      7     8
P2      8    12
P4     12    16
Sequence: P1->P3->P2->P4
○ suresh273@LAPTOP-PI08CKLL:~/OS-Second-Program$
```